Assignment 2: The Socket Exchange

Deadline: 10th September 2026, 23:59 IST

In this assignment, you will build and investigate a very simplified simulation of a trading system. The purpose of the assignment is to learn socket programming and understand how applications communicate over TCP. No prior knowledge of financial markets or electronic trading is required.

The system consists of an Exchange Server, Trader Clients, and Market-Data Clients.

0.1 Exchange Server

The Exchange Server is the central server to which all clients connect using TCP sockets. It receives orders from traders, maintains a simple representation of the market, and sends information back to clients.

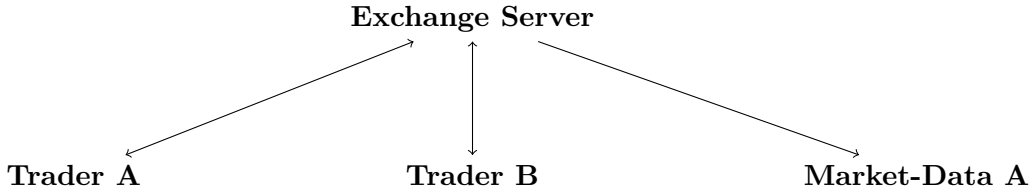
0.2 Trader Clients

A Trader Client represents a participant that can send requests to the Exchange Server and receive responses from it. For example, a trader may submit an order to buy or sell an instrument.

0.3 Market-Data Clients

A Market-Data Client is a read-only client. It does not submit orders. Instead, it connects to the Exchange Server and subscribes to one or more instruments. The server then sends it updates whenever relevant market activity occurs.

Thus, the basic communication pattern is:

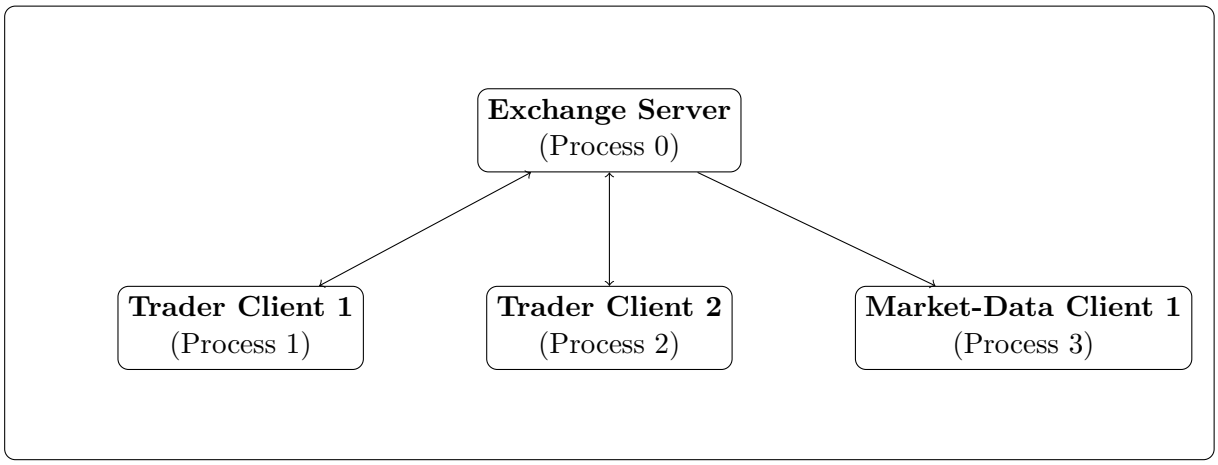


The trading functionality is intentionally minimal. The main focus of this assignment is the network communication underlying the system: TCP connections, sockets, message transmission, multiple concurrent clients, message framing, blocking and non-blocking I/O, connection termination, and related system-level behavior.

1 System Architecture and Communication Model

The entire system will run inside a single FreeBSD virtual machine. The Exchange Server will run as one process, while multiple Trader Clients and Market-Data Clients will run as separate processes within the same VM.

All communication between the clients and the Exchange Server will take place using TCP sockets.



1.1 Exchange Server

The Exchange Server is the central component of the system. It listens for incoming TCP connections and maintains a separate connection with each client.

The server is responsible for:

- receiving requests from Trader Clients;
- maintaining a simple representation of the market;
- sending responses to the appropriate Trader Client;
- sending market-data updates to subscribed Market-Data Clients.

1.2 Trader Clients

A Trader Client represents a participant interacting with the exchange. It can both send requests to and receive messages from the Exchange Server.

For example, a Trader Client may submit an order to buy or sell an instrument and receive an acknowledgement or execution notification in response.

1.3 Market-Data Clients

A Market-Data Client is a read-only client. It does not submit orders. Instead, it subscribes to one or more instruments and receives market-data updates generated by the Exchange Server.

1.4 Communication Model

All clients communicate with the Exchange Server through TCP connections. Clients do not communicate directly with one another.

1.5 TCP Connections

Each client establishes a persistent TCP connection with the Exchange Server.

Each client has its own independent connection.

A TCP connection is bidirectional: both the client and the server can send data at any time.

The Exchange Server must handle multiple client connections concurrently.

1.5.1 Trader Clients

Trader Clients use their TCP connection for both sending and receiving:

```
Trader Client --------> Exchange Server
```

requests / orders

```
Trader Client <-------- Exchange Server
```

responses / notifications

A Trader Client may send an order or request, after which the server may send a response or an asynchronous notification.

1.5.2 Market-Data Clients

Market-Data Clients are read-only with respect to trading activity. They first subscribe to one or more instruments:

```
Market-Data Client --> Exchange Server
```

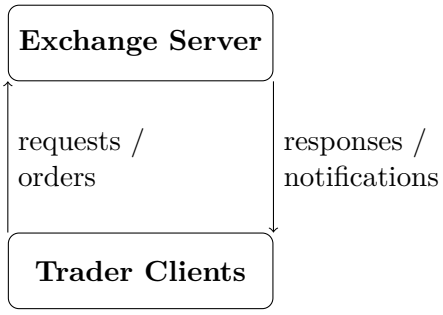
After subscribing, the client does not need to repeatedly request data. The Exchange Server automatically sends relevant updates:

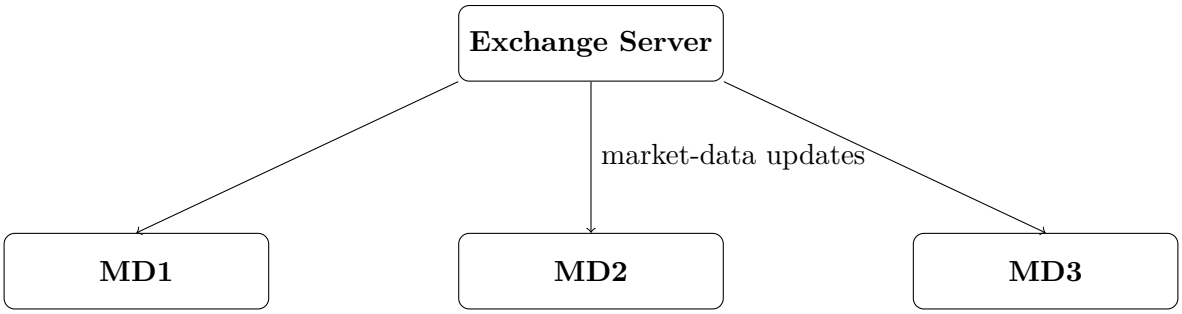
```
Exchange Server ------> Market-Data Client
```

Multiple Market-Data Clients may receive the same update. Thus, the Exchange Server performs one-to-many communication.

1.6 Overall Communication Pattern

The resulting communication patterns are:





The details of the messages exchanged and the exact behavior of orders, subscriptions, and market-data updates are defined in the Protocol Specification section.

The trading logic is intentionally simplified. The primary purpose of the system is to provide a realistic application on top of which you will study TCP sockets, concurrent connections, message framing, blocking I/O, connection management, and related system-level behavior.

2 Protocol Specification

The Exchange Server and its clients communicate using a simple text-based application protocol over TCP. Each application message is a single line terminated by a newline character (\n).

The protocol is intentionally simplified. It models only the basic behavior required for this assignment; no prior knowledge of financial markets is assumed.

2.1 Instruments and Numeric Values

The exchange supports the following two instruments:

```
JNST
IMCT
```

All numeric values transmitted through the protocol are integers.

Quantity and price must be positive integers.

Order IDs generated by the Exchange Server are non-negative integers.

Every accepted order gets a unique ID for as long as the server is running.

Decimal values and negative values are not permitted.

For example:

```
BUY JNST 100 238
SELL IMCT 25 517
```

are valid orders.

2.2 Trader Client Messages

A Trader Client may send the following messages.

2.2.1 Login

```
LOGIN <username>
```

Registers the client with the given username.

Example:

```
LOGIN alice
```

The username must be unique among currently connected Trader Clients.

2.2.2 Buy Order

```
BUY <instrument> <quantity> <price>
```

Requests to buy the specified quantity of an instrument at the specified price.

Example:

```
BUY JNST 100 238
```

2.2.3 Sell Order

```
SELL <instrument> <quantity> <price>
```

Requests to sell the specified quantity of an instrument at the specified price.

Example:

```
SELL JNST 50 238
```

2.2.4 Cancel Order

```
CANCEL <order_id>
```

Requests cancellation of a previously submitted order that still has an unfulfilled quantity.

Example:

```
CANCEL 42
```

2.2.5 Quit

QUIT

Requests a graceful disconnection from the Exchange Server.

2.3 Server Responses to Trader Clients

The Exchange Server may send the following messages to a Trader Client:

```
OK
ERROR <reason>
ORDER_ACCEPTED <order_id>
ORDER_CANCELLED <order_id>
BOUGHT <instrument> <quantity> <price>
SOLD <instrument> <quantity> <price>

OK indicates that a client request was successfully completed. The Exchange Server sends
    OK in response to a successful LOGIN, SUBSCRIBE, or UNSUBSCRIBE request. A successful
     CANCEL request instead results in ORDER_CANCELLED <order_id>.
ORDER_ACCEPTED indicates that an order has been accepted and assigned an order ID. It
    does not necessarily mean that the order has been executed.
BOUGHT indicates that some quantity of an order submitted by this trader has been
    executed as a buy.
SOLD indicates that some quantity of an order submitted by this trader has been executed
    as a sell.
```

These execution notifications may be sent asynchronously. For example, a trader may submit an order, receive ORDER_ACCEPTED, and receive a BOUGHT or SOLD notification much later when a matching order arrives.

2.4 Market-Data Client Messages

A Market-Data Client is read-only with respect to trading. It does not submit or cancel orders.

A Market-Data Client may subscribe to an instrument using:

```
SUBSCRIBE <instrument>
```

Example:

```
SUBSCRIBE JNST
```

It may cancel a subscription using:

```
UNSUBSCRIBE <instrument>
```

Example:

```
UNSUBSCRIBE JNST
```

It may terminate its connection using:

QUIT

2.5 Market-Data Updates

Once a Market-Data Client has subscribed to an instrument, the Exchange Server automatically sends it updates whenever a trade involving that instrument occurs.

The update has the form:

```
TRADE <instrument> <quantity> <price>
```

For example:

```
TRADE JNST 60 238
```

A TRADE message describes what happened in the market. It does not indicate whether the recipient was the buyer or seller.

The same update may be sent to multiple subscribed Market-Data Clients.

2.6 Order Matching

When the Exchange Server receives a valid BUY or SELL order, it first sends:

```
ORDER_ACCEPTED <order_id>
```

The order is then placed in the exchange's order book.

The Exchange Server attempts to match a newly submitted order with existing orders in the order book.

For this assignment, two orders are considered a match only if all of the following are true:

they are for the same instrument;

one is a BUY order and the other is a SELL order;

they have exactly the same price.

When two matching orders are found, the quantity traded is the smaller of the two remaining quantities.

For example:

```
BUY  JNST 100 238
SELL JNST 60  238
```

results in a trade of:

60 JNST at price 238

The buy order then has 40 units remaining.

Any remaining quantity stays in the order book and may be matched by a future order.

If no matching order exists, the newly submitted order simply remains in the order book.

The exchange does not track a trader's cash, holdings, or ability to buy or sell a particular quantity. Traders are assumed to be able to submit any valid order.

2.7 Communication and Asynchronous Notifications

A TCP connection is bidirectional. A client may send messages to the server, while the server may send messages to the client independently of the client's most recent message.

For example:

Trader A Exchange Server

```
BUY JNST 100 238  --------->
                    <------- ORDER_ACCEPTED 42
```

... later, another trader submits

a matching SELL order ...

```
                    <------- BOUGHT JNST 60 238
```

At the same time, Market-Data Clients subscribed to JNST may receive:

```
TRADE JNST 60 238
```

2.8 Client Capabilities

The two types of clients have different roles and therefore support different protocol messages.

Message	Trader Client	Market-Data Client
LOGIN	✓	—
BUY	✓	—
SELL	✓	—
CANCEL	✓	—
SUBSCRIBE	—	✓
UNSUBSCRIBE	—	✓
QUIT	✓	✓

The Exchange Server may send the following messages:

Server Message	Trader Client	Market-Data Client
OK	✓	✓
ERROR	✓	✓
ORDER_ACCEPTED	✓	—
ORDER_CANCELLED	✓	—
BOUGHT	✓	—
SOLD	✓	—
TRADE	—	✓

A Trader Client does not subscribe to market data. It receives notifications only about its own orders and executions.

A Market-Data Client is read-only with respect to trading. It cannot submit or cancel orders. It receives TRADE notifications for instruments to which it has subscribed.

```
TRADE is therefore intended for Market-Data Clients, whereas BOUGHT and SOLD are private
    execution notifications sent only to the traders whose orders were executed.
```

2.9 Message Framing

TCP provides a continuous byte stream, not a sequence of application-level messages. Therefore, the protocol does not assume that one call to send() corresponds to one call to recv(), or that one recv() returns exactly one application message.

A receiver must correctly handle cases where:

- one message is received through multiple recv() calls;
- multiple messages are received through a single recv() call; or
- a message is split at an arbitrary position in the TCP stream.

The newline character (\n) marks the end of an application-level message.

Correctly handling message framing is an important part of the assignment.

3 Assignment Tasks

The assignment is divided into several stages. You will progressively build the components of the trading system and use them to investigate how applications communicate using TCP sockets.

3.1 Implement the Exchange Server

Implement an Exchange Server that:

- accepts and manages multiple simultaneous TCP client connections;
- distinguishes between Trader Clients and Market-Data Clients;
- process all messages permitted for that client type according to the Protocol Specification and deliver responses and notifications to the appropriate clients.
- maintain the set of outstanding orders, perform the specified order-matching and partial-execution behavior,

3.2 Implement the Clients

Implement both types of clients:

Trader Client

A Trader Client should allow a user to connect to the Exchange Server, submit and cancel orders, and receive responses and execution notifications.

Market-Data Client

A Market-Data Client should allow a user to connect to the Exchange Server, subscribe to instruments, and receive market-data updates automatically when trades occur.

3.3 Support Multiple Simultaneous Clients

The Exchange Server must support multiple Trader and Market-Data Clients connected at the same time.

Clients may send messages independently, and the server must continue to service other clients even when one client is idle or otherwise not actively communicating.

3.4 Investigate and Test the System

After implementing the system, you will perform a series of experiments designed to investigate its behavior as a TCP-based application.

These experiments will involve creating different communication patterns, observing the behavior of the clients and server, and using appropriate system and network tools to understand what is happening.

The experiments will focus on topics such as:

- TCP connections and connection termination;
- application-level message framing;
- partial reads and writes;
- multiple simultaneous connections;
- blocking and non-blocking communication;
- server-to-client notifications and broadcasting.

The specific experiments and questions will be provided as part of the assignment.

4 Implementation Requirements

The following requirements apply to the Exchange Server and both types of clients.

4.1 Programming Language and Networking API

The Exchange Server and both types of clients must be implemented in either C or Python.

Regardless of the language, your implementation must communicate using TCP sockets directly through the language's standard or system-level socket API.

The following socket operations must be performed using the underlying socket interface:

- creating a socket (socket);
- binding a server socket to an address and port (bind);
- placing a server socket in the listening state (listen);
- accepting incoming connections (accept);
- establishing a client connection (connect);
- sending and receiving data (send() / write(), recv() / read(), or their direct Python equivalents);
- closing a connection (close);
- performing connection shutdown when required (shutdown).

The exact function names may differ between C and Python, but the implementation must directly use the corresponding socket operations.

4.1.1 Allowed

The following are allowed:

- C and Python standard-library/system-level socket APIs, including:
- POSIX socket functions in C;
- Python's socket module.
- Standard OS-level I/O and socket mechanisms such as:
- select();
- poll();
- kqueue().
- Operating-system and network-analysis tools such as tcpdump, sockstat, netstat, procstat, and similar tools for testing and debugging.

4.1.2 Not Allowed

You may not use any high-level networking framework or library that abstracts away the socket-level programming required by this assignment.

In particular, do not use:

- networking frameworks such as Boost.Asio, libevent, libuv, or equivalent libraries;
- high-level asynchronous networking frameworks such as Python's asyncio;
- HTTP, WebSocket, RPC, or web-server frameworks for communication;
- third-party libraries that provide connection management, event loops, message framing, or other networking functionality on top of the socket API;
- any library whose purpose is to substantially hide the TCP socket operations listed above.
- For Python, the use of the standard socket module is permitted, but higher-level networking frameworks built on top of it are not.
- For C, direct use of the POSIX socket API is permitted, but networking libraries that abstract away the socket API are not.

The purpose of these restrictions is to ensure that you directly learn and use the TCP socket programming interface rather than relying on a framework that implements the networking functionality for you.

If you are unsure whether a particular library is permitted, do not use it until you have confirmed its use with the instructor.

4.2 Protocol Compliance

All messages exchanged between clients and the Exchange Server must conform to the Protocol Specification given in Section 2.

The implementation must not assume that there is a one-to-one correspondence between calls to send() and recv(). A single application message may be split across multiple recv() calls, and multiple application messages may be returned by a single recv() call. The implementation must use the specified newline delimiter to reconstruct application-level messages.

4.3 Multiple Clients

The Exchange Server must support multiple clients connected simultaneously, including both Trader Clients and Market-Data Clients.

The server must support at least 10 simultaneously connected clients, including at least 2 Trader Clients and at least 4 Market-Data Clients.

The server must not depend on a fixed number of clients.

4.4 Correct Connection Handling

The implementation must correctly handle normal connection establishment and termination, including clients that disconnect unexpectedly.

A failure or disconnection of one client should not cause the server to terminate or prevent it from servicing other connected clients.

Persistent connections: a client can issue multiple protocol messages over the same TCP connection; it should not need to reconnect for every operation.

4.5 Client Roles

The implementation must enforce the distinction between the two client types described in the protocol:

Trader Clients may submit and cancel orders.

Market-Data Clients may subscribe to instruments and receive market-data updates.

A client must not be able to use commands that are not permitted for its role.

4.6 No Trading-State Restrictions

The exchange does not maintain trader balances, holdings, or other financial constraints. A Trader Client may submit any syntactically valid order permitted by the protocol.

The implementation should focus on the communication and order-matching behavior specified by the assignment rather than realistic financial-system features.

4.7 Implementation Freedom

Unless explicitly stated elsewhere in the assignment, the choice of concurrency and I/O mechanism is left to you. You may use an approach such as threads, select(), poll(), or kqueue().

You must, however, understand and be able to explain the behavior of the mechanism you choose.

5 Development Environment and Setup

Use a virtualization platform capable of running FreeBSD 14.4-RELEASE or a later compatible version on your host architecture. VirtualBox is recommended for x86-64 systems. Students using Apple Silicon Macs may use UTM, VMware Fusion, or another suitable ARM64-capable virtualization solution.

5.0.1 Requirements

- One VM running FreeBSD 14.4-RELEASE (preferred) or a later compatible FreeBSD release. A configuration such as 2 CPU cores and at least 2 GB RAM should be sufficient. A graphical/X Window environment is not required.
- The Exchange Server, Trader Clients, and Market-Data Clients must all run as separate processes inside the same FreeBSD VM.
- The VM must have a working network configuration that allows the client processes to communicate with the Exchange Server using TCP. Since all components run within the same VM, the loopback interface may be used.
- You are responsible for installing and configuring the compiler, Python, libraries, and other software required by your implementation and experiments.
- You may use appropriate FreeBSD documentation, manuals, and other resources to configure and administer your VM.

5.0.2 Setup

You should configure your FreeBSD VM before beginning the implementation. You are expected to be comfortable starting the Exchange Server and multiple client processes simultaneously and observing their network communication.

The assignment will use the VM both for implementing the trading system and for performing the experiments described in Section 6.

5.1 Network Configuration

The Exchange Server must listen on a TCP port that is accessible to the client processes running in the same VM.

Clients must connect to the server using a TCP address and port. You may use the loopback interface for communication between the processes.

For example:

Exchange Server: 127.0.0.1:<port>

Trader Client: connects to the Exchange Server

Market-Data Client: connects to the Exchange Server

The specific port number and network configuration are left to you.

5.2 Running Multiple Clients

The server must be capable of handling multiple clients simultaneously. You should therefore be able to run multiple client processes at the same time.

For example:

```
./exchange_server
./trader_client
./trader_client
./market_data_client
./market_data_client

...
```

The clients may all run on the same VM and connect independently to the Exchange Server.

5.3 Development and Debugging

You are expected to use the operating-system and network tools available on FreeBSD to understand and debug your implementation.

In particular, you should become familiar with tools such as:

sockstat

netstat

tcpdump

procstat

ktrace

These tools will be useful during the experiments described later in the assignment.

6 Experiments and Investigation

This section contains a series of hands-on experiments designed to help you understand the behavior of TCP sockets and the interaction between your application and the operating system.

A single Python script, experiment.py, is provided for all experiments. You must specify the experiment number as a mandatory command-line argument:

```
python3 experiment.py <experiment_number>
```

For example:

```
python3 experiment.py 1
```

Each experiment creates a specific networking scenario for you to investigate. You should use the network and system tools available on FreeBSD to observe what is happening and answer the question associated with the experiment.

For every experiment:

- Run experiment.py with the specified experiment number.

- Use the indicated tools to investigate the resulting behavior.
- Answer the question given in the experiment.
- Include the required deliverable in your assignment report.

The experiments are intended to encourage investigation rather than simply following a fixed sequence of commands. You may use additional tools or perform additional experiments whenever they help you understand the observed behavior.

6.0.1 Deliverables

Each experiment has a specified deliverable, typically consisting of:

- terminal screenshot(s) showing the relevant observation; and
- a short answer to the question posed by the experiment.

The screenshot should contain enough information to establish the observation on which your answer is based. Your answer should explain the observation in terms of the socket and TCP behavior being investigated.

The experiments build upon one another, so you are encouraged to complete them in order.

6.0.2 Submission Interface

To allow the provided experiment script to run your implementation without depending on the programming language you have chosen, your submission must provide standard launcher scripts for the Exchange Server and the two types of clients.

The launcher scripts must be placed in the server/ and client/ directories directly under the root of your submission, as specified by the submission directory structure given at the end of the assignment.

Exchange Server Launcher

The file server/run-server must start your Exchange Server.

For a C implementation, it could contain:

```
#!/bin/sh
exec ./exchange_server "$@"
```

For a Python implementation, it could contain:

```
#!/bin/sh
exec python3 server.py "$@"
```

The launcher must use exec so that the launcher process is replaced by the actual server process.

Trader Client Launcher

The file client/run-trader must start your Trader Client.

For example:

```
#!/bin/sh
exec python3 trader.py "$@"
```

or:

```
#!/bin/sh
exec ./trader_client "$@"
```

Market-Data Client Launcher

The file client/run-market-data must start your Market-Data Client.

For example:

```
#!/bin/sh
exec python3 market_data.py "$@"
```

or:

```
#!/bin/sh
exec ./market_data_client "$@"
```

Launcher Requirements

- The launcher scripts must satisfy the following requirements:
- They must be executable.
- They must use exec to launch the corresponding program.
- They must pass all command-line arguments they receive to the underlying program.
- server/run-server must start the Exchange Server and keep running while the server is running.
- client/run-trader and client/run-market-data must start the corresponding client programs.
- Your implementation must not require the experiment script to know which programming language or executable is being used.

For example, the experiment script may invoke:

```
./server/run-server 127.0.0.1 5000
```

without knowing whether the server is implemented in C or Python.

Similarly, it may invoke:

```
./client/run-trader 127.0.0.1 5000 alice
```

or:

```
./client/run-market-data 127.0.0.1 5000 JNST
```

The exact arguments provided to these launchers will be specified by the individual experiments.

6.1 Experiment 1 — Listening and Connected Sockets

Run:

```
python3 experiment.py 1
```

Investigate:

The experiment will start the Exchange Server and establish a TCP connection from a client.

Question:

After the client has connected, identify the TCP sockets associated with the Exchange Server. How does the listening socket differ from the socket representing the connection to the client?

Deliverable:

Submit terminal screenshot(s) clearly showing the relevant output, with the sockets associated with the Exchange Server identifiable, together with a one- or two-sentence answer to the question.

6.2 Experiment 2 — Observing TCP Connection States

Run:

```
python3 experiment.py 2
```

Investigate:

The experiment will establish a TCP connection between a Trader Client and the Exchange Server. You will be given opportunities to observe the connection while it is being established, while it is active, and after one of its endpoints terminates the connection.

Use appropriate FreeBSD network tools to observe the TCP connection at these different stages. Pay attention to the local and remote endpoints and to the TCP state reported for the connection.

Question:

How does the TCP state of the client-server connection change during its lifetime, and what events cause the observed state changes?

Deliverable:

Submit terminal screenshot(s) showing the relevant observations at different stages of the connection, together with a concise description of the state changes and the events that caused them.

6.3 Experiment 3 — TCP as a Byte Stream

Run:

```
python3 experiment.py 3
```

Investigate:

The experiment will establish a TCP connection to the Exchange Server and send a single valid application-level message in several small pieces rather than as one complete transmission.

Investigate what the Exchange Server receives and how it processes the message. You may use any appropriate FreeBSD tools or modify your implementation to make the relevant behavior observable.

Question:

Does the Exchange Server receive the application-level message as one complete unit, or can the message be received in multiple pieces? Explain what this demonstrates about the relationship between TCP and application-level message boundaries.

Deliverable:

Submit terminal screenshot(s) clearly showing the relevant observation from your experiment, together with a one- or two-sentence answer to the question.

6.4 Experiment 4 — One Client Should Not Stall the Others

Run:

```
python3 experiment.py 4
```

Investigate:

The experiment will start the Exchange Server and establish two TCP connections. The first client will connect and then remain completely idle—it will not send any application-level data. Shortly afterwards, a second client will attempt to connect and communicate with the server.

Investigate how the Exchange Server behaves when one connected client is idle while another client is attempting to communicate. Use appropriate system/network tools to determine where the server is spending its time and what is happening to the two TCP connections.

You should investigate beyond the visible client behavior: determine whether the server is blocked, and if so, identify the operation responsible for the blocking.

Question:

When Client 1 remains connected but sends no data, can the Exchange Server still accept and service Client 2? Use your observations to identify the server operation that determines the answer.

Deliverable:

Submit terminal screenshot(s) showing sufficient system/network evidence to support your answer, together with a one- or two-sentence explanation of why the presence of an idle client does or does not prevent the server from servicing another client.

6.5 Experiment 5 — Multiple Clients and I/O Multiplexing

(Optional, not mandatory)

Run:

```
python3 experiment.py 5
```

Investigate:

The experiment will start the Exchange Server and establish several simultaneous TCP connections. Some clients will remain idle, while others will generate activity at different times. During the experiment, inspect the running server and its connections using appropriate FreeBSD system and network tools.

Your goal is to determine how the server reacts when multiple sockets are simultaneously present but only some of them are ready for communication.

Question:

At a particular point during the experiment, which client connections are actually ready for the server to service, and what evidence from the running system allows you to determine this?

Deliverable:

Submit terminal screenshot(s) showing the relevant observation(s) from a FreeBSD tool, together with a one- or two-sentence answer identifying which connections were ready and how you determined this.

6.6 Experiment 6 — FIN vs. RST: Orderly and Abrupt Connection Termination

Run:

```
python3 experiment.py 6
```

Investigate:

The experiment will establish a TCP connection between a Trader Client and the Exchange Server. The client will then terminate the connection abruptly, without performing an orderly shutdown.

Investigate how this termination differs from the orderly termination you observed in Experiment 2. Use appropriate FreeBSD tools to observe both the TCP packets exchanged and the state observed by the Exchange Server.

Pay particular attention to what the server's socket observes and how the TCP connection changes during the termination.

Question:

How does an abrupt client termination differ from the orderly shutdown in Experiment 6? Identify the TCP event observed on the network and the corresponding behavior of the Exchange Server's socket.

Deliverable:

Submit terminal screenshot(s) showing the key observations from your investigation, together with a concise description of the steps and tools you used to distinguish the two types of termination.

6.7 Experiment 7 — Backpressure and the Slow Receiver

Run:

```
python3 experiment.py 7
```

Investigate:

The experiment will start the Exchange Server with multiple Market-Data Clients. One client will deliberately read market-data updates very slowly, while another client will continue reading normally.

The experiment will generate a sustained stream of market-data updates. Investigate how the TCP connection to the slow client behaves as the client stops reading, and whether the slow client eventually affects the server's ability to communicate with the other clients.

Use appropriate FreeBSD network tools to observe the connections and traffic generated during the experiment. You should compare the behavior of the connections to the slow and normal clients and look for evidence of changes in communication as the experiment progresses.

Question:

How does the Exchange Server's TCP connection to the slow client behave as the client stops reading, and what evidence shows whether this eventually affects the server's ability to communicate with other clients?

Deliverable:

Submit terminal screenshot(s) showing the key network-level evidence from your investigation, together with a concise description of the steps and tools you used to arrive at your answer.

6.8 Experiment 8 — Unexpected Client Disconnection

Run:

```
python3 experiment.py 8
```

Investigate:

The experiment will establish a TCP connection between a Market-Data Client and the Exchange Server. The client will subscribe to an instrument and begin receiving market-data updates. While the server is actively communicating with the client, the client will be terminated unexpectedly.

Investigate what happens to the TCP connection after the client disappears. Compare the affected connection with another client that remains connected, and examine the network traffic surrounding the disconnection.

Question:

When a client disappears unexpectedly while the Exchange Server is communicating with it, what happens to their TCP connection, and what network-level evidence allows you to determine how the server detects the failure?

Deliverable:

Submit terminal screenshot(s) showing the key network-level evidence surrounding the disconnection, together with a concise description of the steps and tools you used to determine what happened.

6.9 Bonus — Connection Scalability and I/O Design

Extend your Exchange Server so that it can maintain at least 70,000 simultaneously established idle TCP connections.

As part of this bonus, you must also write a client-generation program that creates and maintains the required number of simultaneous TCP connections to your Exchange Server. The program should keep these connections established without exchanging application-level data.

As the number of idle connections increases, measure the following resources and record your observations:

Number of idle connections	Server memory usage	Server CPU usage	Open file descriptors for server	System-wide open file descriptors	Socket-buffer usage/limit	Maximum connections successfully established
10,000						
20,000						

Number of idle connections	Server memory usage	Server CPU usage	Open file descriptors for server	System-wide open file descriptors	Socket-buffer usage/limit	Maximum connections successfully established
30,000						
40,000						
50,000						
60,000						
70,000						

Use appropriate FreeBSD tools and system interfaces to obtain these measurements. You should ensure that your measurements are taken while the specified number of TCP connections are simultaneously established and idle.

After completing the table, answer the following questions:

1. At each connection count, is your server able to maintain all requested connections? If not, identify the connection count at which it first fails.
2. What is the first significant bottleneck encountered by your implementation? Use the measurements in your table to support your conclusion. The bottleneck may be memory, CPU, file-descriptor limits, socket-buffer limits, or another OS resource.
3. How does your concurrency/I/O design contribute to the observed bottleneck? Explain whether your implementation creates a separate thread/process for each connection or uses an I/O-multiplexing mechanism such as poll() or kqueue().
4. Would changing the I/O/concurrency mechanism help with the bottleneck you identified? For example, would replacing a thread-per-connection design with poll() or kqueue() reduce the relevant resource consumption? Explain why or why not.
5. Quantify the trade-off. If you change your implementation or configuration, measure the relevant resource again and compare it with your original design.

6.9.1 Bonus Deliverables

Submit the following:

1. The client-generation program used to create and maintain the TCP connections.
2. The completed resource-measurement table. (Include inside the Experiment Report)
3. Screenshots of the FreeBSD commands and their outputs used to obtain the table entries for 10,000, 40,000, and 70,000 idle connections. Each screenshot must clearly show the number of active connections and the corresponding measurements. (Include inside the Experiment Report)
4. A concise analysis answering the questions above. (Include inside the Experiment Report)

The purpose of this bonus is to investigate the relationship between TCP connection identity, per-connection resource consumption, operating-system limits, and the scalability of different approaches to handling large numbers of mostly idle sockets.

7 Submission Structure and Deliverables

7.1 Submission File

Submit one ZIP file containing the complete submission.

The automated submission checker is strict about the filename. Your team is identified by the two roll numbers of its members.

Sort the two roll numbers alphabetically and use that order in the filename, regardless of which team member uploads the submission.

For example, if the roll numbers are:

2024CS10057

2024CS10093

the filename must be:

`A2_2024CS10057_2024CS10093.zip`

The naming rules are:

- Roll numbers must be in exactly the same form as they appear on the LMS.
- Use capital letters where applicable.
- Do not add spaces or hyphens.
- Always use the alphabetically sorted order of the two roll numbers.
- Only one team member should upload the ZIP file.

Your submission must follow the directory structure specified below. The purpose of this structure is to allow the provided experiment script to run your implementation without depending on the programming language or implementation details you have chosen.

```
A2_2024CS10057_2024CS10093.zip/
+-- server/
|   +-- run-server
|
+-- client/
|   +-- run-trader
|   +-- run-market-data
|
+-- src/
|   +-- <your source code>
|
+-- Makefile (optional)
+-- README.md
+-- report.pdf
```

The exact contents of src/ and the build system are left to you, subject to the implementation requirements given earlier.

7.2 Server and Client Launchers

The following launcher files are mandatory:

```
server/run-server
client/run-trader
client/run-market-data
```

They must satisfy all requirements specified in Section 4.2.

7.3 Source Code

Include all source code and other files required to build, prepare, and run your implementation.

A Makefile is optional. Use an appropriate build or dependency-management mechanism for your chosen language where necessary.

Your submission must not depend on files outside the submission directory.

7.4 README

The README.md must contain the information necessary to build or prepare and run your implementation.

At minimum, specify:

- the programming language and compiler/runtime used;
- the commands required to build or prepare the Exchange Server and clients, if applicable;
- how to start the Exchange Server;
- how to start each type of client;
- any configuration required for your implementation.

7.5 Submission Independence

Your submission must be self-contained and reproducible on a correctly configured FreeBSD environment.

Do not assume that the evaluator has installed software, configured environment variables, or created files that are not documented in your README.

7.6 Deadline and Late Submission

The submission is due by: **10th September 2026, 23:59 IST**

Late submissions are subject to the following penalties:

- 20% of the total marks per day late, for up to 3 days.
- After 3 days, the submission portal will close and no further submissions will be accepted.
- A ZIP file that fails the automated submission-structure check will receive a 10% penalty and the team will be given one opportunity to correct and resubmit the ZIP file.

8 Experiment Report

Submit a single report containing your responses to all required experiments.

There is no prescribed format for answering the questions in the individual experiments. Your response to each experiment should be clear and concise and should demonstrate how you arrived at your answer.

For each experiment, include:

- your answer to the question;
- a brief description of the approach you took to investigate the problem;
- the specific commands and tools you used;
- the terminal screenshot(s) requested by the experiment.

The screenshots should clearly show the relevant observations or command output supporting your answer. You may include additional screenshots or observations when they are useful for explaining your investigation.

8.1 Implementation Decisions

The report should also contain a short section describing the major implementation decisions you made.

In particular, state:

- the concurrency/I/O approach used by your Exchange Server, such as threads, select(), poll(), or kqueue();
- why you chose this approach;
- any other significant design decisions that affect how your server handles multiple clients or communicates over TCP.

Keep this section concise and focus on decisions relevant to the networking aspects of the assignment.

9 Grading

The assignment will be graded on both the correctness of your implementation and your understanding of TCP socket programming demonstrated through the experiments and the accompanying report.

Component	**Weight**
Exchange Server and Client Implementation	35%
Experiments and Investigation	40%
Experiment Report and Implementation Decisions	5%
Viva	20%
Bonus	10%
Total	**110%**

9.1 Implementation — 35%

This component evaluates whether your Exchange Server and clients correctly implement the required functionality and conform to the protocol specification.

The implementation will be evaluated on aspects such as:

correct protocol behavior;

support for multiple simultaneous clients;

correct order matching and execution;

correct market-data subscriptions and notifications;

correct handling of TCP communication and connection termination.

9.2 Experiments and Investigation — 40%

This component evaluates the quality of your observations and investigations across the required experiments.

Marks will be awarded for:

correctly answering the questions posed by the experiments;

using appropriate network/system tools to investigate the behavior;

presenting relevant observations and evidence;

The correctness of the implementation alone is not sufficient to obtain full marks in this component.

9.3 Experiment Report and Implementation Decisions — 5%

This component evaluates the clarity and completeness of the submitted report, including the required screenshots, investigation steps, commands/tools used, and the discussion of your implementation decisions.

Particular attention will be given to your explanation of the concurrency/I/O mechanism chosen for your Exchange Server and the reasoning behind that choice.

9.4 Viva — 20%

This part is intended to assess your understanding of the concepts underlying your implementation.